

Name: _____ Score: _____

ECE 2036 Test 2

Instructions

1. You may have a TWO 8 ½ x 11 sheets of paper with **HANDWRITTEN** information on the front and back to use during the test.
2. You do not need a calculator.
3. You must turn off your cell phone.
4. If you need to go to the restroom, you must ask the proctor for permission AND leave your cell phone with the proctor of the exam.

Problem 1 (15 points) Given the following code, please CIRCLE the lines of code in main() that cause a compile error.

```
#include <iostream>

using namespace std;

class Trees
{
public:
    float height;
    float latitude;
    float longitude;
};

class Pine: public Trees
{
public:
    int numPineCones;
};

class Maple: public Trees
{
public:
    float syrupCapacity;
};

class CrepeMyrtle: public Trees
{
public:
    int daysBetweenPruning;
};

int main()
{
    Trees * TreePtr;

    Pine * PinePtr;

    Maple * MaplePtr;

    const CrepeMyrtle * CrepePtr;

    TreePtr = new Pine;

    MaplePtr = TreePtr;

    CrepePtr = new CrepeMyrtle;

    PinePtr = TreePtr;

    CrepePtr->daysBetweenPruning = 50;

    TreePtr->height = 11.2;
}
```

Problem 2 (15 points): Please consider the following overload of the postfix operator that we talked about in class. This is for the time class, which has its basic class interface in the Appendix A of this test.

```
Time Time::operator++(int)
{
    Time temp = *this;
    minute ++;
    return temp;
}
```

I would like for you to change this member function so that the following statement does not cause a compile error.

```
(timeObject++)++;
```

To get this to work you have some freedom in its operation. My only stipulation is that this ++ overload "generally" increments the time object's minutes by one every time it is called.

PLEASE EXPLICITLY USE THE this POINTER IN ALL APPROPRIATE STATEMENTS TO RECEIVE FULL CREDIT FOR THIS PROBLEM... DO NOT RELY ON IMPLICIT USE BY THE COMPILER.

//show your new implementation of the postfix operator here...

Problem 3 (30 points): Please consider the following `main()` function. The classes `Top`, `Object1`, `Object2`, and `Object3` are a part of a class hierarchy that you will need to design in part (b).

```
int main()
{
    vector<Top*> topVector(3, NULL);

    topVector[0] = new Object1;
    topVector[1] = new Object2;
    topVector[2] = new Object3;

    for (Top * value : topVector)
    { value->printGreetings(); }

    PreventMemoryLeaks(topVector);
}
```

(a) Please show the implementation of the global member function `PreventMemoryLeaks`, which as the name implies, needs to make sure that this program does not have memory leaks! PLEASE USE ITERATORS IN YOUR SOLUTION TO RECEIVE FULL CREDIT FOR THIS PROBLEM.

(b) Please provide the bare minimum code of each class definition (Top, Object1, Object2, and Object3) to get the code working in `main()` on the previous page such that the output of the program is given by:

```
Hello from Object1!  
Hello from Object1!  
Hello from Top!
```

Make sure you choose an appropriate hierarchy and the appropriate implementations of the member function `printGreetings()`. An "appropriate" implementation of `printGreetings()` means that if a class has a direct implementation of `printGreetings()` that it says its hello from that class. For example, the `printGreetings()` in Top class would say "Hello from Top!" An implementation of `printGreetings()` in Object1 class would, for example, say "Hello from Object1!"

Problem 4 (20 points): What is the output of the following program? Please box your final answer.

```
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main()
{
    map <char,char> structure1 ;

    for (int i=0; i<26; i++)
    {
        structure1.insert(make_pair(static_cast <char>(97+i),static_cast <char> (97+(i+3)%26)));
    }

    string input = "toeceornottoecez";
    cout << input << endl;

    for (int i=0; i < input.size(); i++)
    {
        input[i] = structure1[input[i]];
    }
    cout << input << endl;
}
```

Problem 5 (20 points): Please trace through the code and indicate the output of this program. Please put a BOX around your answer.

```
#include <iostream>
using namespace std;

class A
{
    public:

    A(int a): dataA(a) {cout << "Constructor " << dataA << endl;}
    A(const A & Acopy) { dataA = Acopy.dataA; cout << "CopyC with " << dataA << endl;}
    ~A(){cout << "Destructor "<< dataA << endl;}
    int getDataA() const {return(dataA);}
    void IncrementAPlus() {dataAPlus++;}
    void PrintData () const { cout << dataA << " " << dataAPlus << endl;}

    private:
    int dataA;
    static int dataAPlus;
};

int A::dataAPlus = 1;

void FunctionCall(A Aobject)
{
    cout << "In function with " << Aobject.getDataA() << endl;
}

int main()
{
    A object1(10);
    A object2(20);

    FunctionCall(object1);
    FunctionCall(object2);
    object2.IncrementAPlus();
    object1.PrintData();
    object1.IncrementAPlus();
    object2.PrintData();

    }//end main
```

Appendix A: Basic Time Class Interface

```
class Time
{
public:
    Time(int=0,int=0,int=0);

    //set functions
    void setTime(int,int,int);
    void setHour(int);
    void setMinute(int);
    void setSecond(int) ;

    //get functions
    int getHour() const;
    int getMinute() const;
    int getSecond() const;

    //general member functions
    void printTime() const;

private:
    int hour; //0-23 hours
    int minute; //0 to 59
    int second; //0 to 59

}; //end of time class
```

ORDER OF PRECEDENCE CHART

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right
2	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	. ->	Element selection by reference Element selection through pointer	
3	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	new, new[] delete, delete[]	Dynamic memory allocation Dynamic memory deallocation	
4	.* ->*	Pointer to member	Left-to-right
5	* / %	Multiplication, division, and remainder	Left-to-right
6	+ -	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
9	== !=	For relational = and ≠ respectively	
10	&	Bitwise AND	
11	^	Bitwise XOR (exclusive or)	
12		Bitwise OR (inclusive or)	
13	&&	Logical AND	
14		Logical OR	
15	?:	Ternary conditional ^[1]	Right-to-left
	=	Direct assignment (provided by default for C++ classes)	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
16	&= ^= =	Assignment by bitwise AND, XOR, and OR	Left-to-right
17	throw	Throw operator (for exceptions)	
17	,	Comma	Left-to-right

ASCII CHART

Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char
0x00	0	NULL null	0x20	32	Space	0x40	64	@	0x60	96	~
0x01	1	SOH Start of heading	0x21	33	!	0x41	65	A	0x61	97	a
0x02	2	STX Start of text	0x22	34	"	0x42	66	B	0x62	98	b
0x03	3	ETX End of text	0x23	35	#	0x43	67	C	0x63	99	c
0x04	4	EOT End of transmission	0x24	36	\$	0x44	68	D	0x64	100	d
0x05	5	ENQ Enquiry	0x25	37	%	0x45	69	E	0x65	101	e
0x06	6	ACK Acknowledge	0x26	38	&	0x46	70	F	0x66	102	f
0x07	7	BELL Bell	0x27	39	'	0x47	71	G	0x67	103	g
0x08	8	BS Backspace	0x28	40	(	0x48	72	H	0x68	104	h
0x09	9	TAB Horizontal tab	0x29	41	)	0x49	73	I	0x69	105	i
0x0A	10	LF New line	0x2A	42	*	0x4A	74	J	0x6A	106	j
0x0B	11	VT Vertical tab	0x2B	43	+	0x4B	75	K	0x6B	107	k
0x0C	12	FF Form Feed	0x2C	44	,	0x4C	76	L	0x6C	108	l
0x0D	13	CR Carriage return	0x2D	45	-	0x4D	77	M	0x6D	109	m
0x0E	14	SO Shift out	0x2E	46	.	0x4E	78	N	0x6E	110	n
0x0F	15	SI Shift in	0x2F	47	/	0x4F	79	O	0x6F	111	o
0x10	16	DLE Data link escape	0x30	48	0	0x50	80	P	0x70	112	p
0x11	17	DC1 Device control 1	0x31	49	1	0x51	81	Q	0x71	113	q
0x12	18	DC2 Device control 2	0x32	50	2	0x52	82	R	0x72	114	r
0x13	19	DC3 Device control 3	0x33	51	3	0x53	83	S	0x73	115	s
0x14	20	DC4 Device control 4	0x34	52	4	0x54	84	T	0x74	116	t
0x15	21	NAK Negative ack	0x35	53	5	0x55	85	U	0x75	117	u
0x16	22	SYN Synchronous idle	0x36	54	6	0x56	86	V	0x76	118	v
0x17	23	ETB End transmission block	0x37	55	7	0x57	87	W	0x77	119	w
0x18	24	CAN Cancel	0x38	56	8	0x58	88	X	0x78	120	x
0x19	25	EM End of medium	0x39	57	9	0x59	89	Y	0x79	121	y
0x1A	26	SUB Substitute	0x3A	58	:	0x5A	90	Z	0x7A	122	z
0x1B	27	FSC Escape	0x3B	59	;	0x5B	91	[	0x7B	123	{
0x1C	28	FS File separator	0x3C	60	<	0x5C	92	\	0x7C	124	
0x1D	29	GS Group separator	0x3D	61	=	0x5D	93	]	0x7D	125	}
0x1E	30	RS Record separator	0x3E	62	>	0x5E	94	^	0x7E	126	~
0x1F	31	US Unit separator	0x3F	63	?	0x5F	95	_	0x7F	127	DEL